ADAM (A Decentralized Approach Model) Cooperative Consensus

1. Introduction and Background

The **ADAM (A Decentralized Approach Model)** consensus mechanism is a cooperative hybrid consensus model designed to harden the blockchain against mining centralization, selfish mining, block grinding, and targeted Denial-of-Service (DoS) attacks on block leaders.

Traditional Proof-of-Work (PoW) and Proof-of-Stake (PoS) protocols suffer from distinct vulnerabilities:

- In PoW, miners with massive hashing power dominate block production, leading to pools that centralize consensus.
- In PoS, the richest staking wallets have a disproportionate chance of producing blocks.
- In both models, if the identity of the next block producer is known beforehand (pre-elected), they become targets for DoS attacks or bribery. If block contents can be manipulated to alter the next block's randomness, miners can perform *grinding attacks* to skew future leader elections.

ADAM addresses these issues by splitting the block production process into a **Cooperative Hybrid Round**:

1. **Election Phase**: For each block slot, a pseudo-random seed deterministically elects a pool of N **Miners** and **1 Coordinator**.
2. **Solving Phase**: The elected Miners compile and solve lightweight, parallelized PoW puzzles (partial solutions) based on the current block's target.
3. **Aggregation Phase**: The elected Coordinator collects the partial solutions, verifies them, packages them into a block template, signs the final block header using their private key, and broadcasts it to the network.

---

2. Consensus Architecture & Parameters

ADAM consensus is activated conditionally based on block height. The core parameters are defined in `src/consensus/params.h` and configured per network in `src/chainparams.cpp`.

To allow the network to bootstrap smoothly when the active masternode count is low, ADAM supports two modes of operation controlled dynamically by a network Spork.

A. Fallback Mode (Block Version 11)

Fallback Mode is designed for the bootstrap phase of the network. It operates without requiring active Masternode registration and is fully self-contained.

- **Miners Count (N)**: Dynamically determined by the size of the key pool $K - 1$, where $11 \le K \le 14$ (keys are deterministic and the elected Coordinator is appended as the last element of `vAdamMiners`).
- **Consensus Threshold (T)**: Fixed at the consensus parameter `nAdamThreshold` (which is `7` on Mainnet/Regtest, but set to `3` on Testnet to facilitate local testing/block coordination) in both modes.

- **Self-Contained Header Layout**: `vAdamMiners` stores all K public keys (first $K-1$ elected miners, last key is the coordinator). `vAdamSolutions` stores the $K-1$ partial solutions.

B. Standard Mode (Block Version 12)

Standard Mode represents the full cooperative consensus state, requiring a fully populated Masternode network.

- **Miners Count (N)**: Configured via the consensus parameter `nAdamMinersCount` (defined in `src/consensus/params.h` and initialized in `src/chainparams.cpp` to `11` on Mainnet/Testnet/Regtest).
- **Consensus Threshold (T)**: Configured via the consensus parameter `nAdamThreshold` (defined in `src/consensus/params.h` and initialized in `src/chainparams.cpp` to **`7`** on Mainnet/Regtest, and **`3`** on Testnet).
- **Elected Coordinator**: The coordinator is elected dynamically from the active Masternode list and is distinct from the miners list.

C. Spork-Controlled Activation (`SPORK_21_ADAM_STANDARD_MODE`)

The transition between Fallback Mode (Version 11) and Standard Mode (Version 12) is controlled by `SPORK_21_ADAM_STANDARD_MODE` (Spork ID `10020`).

- **Default Value**: `4070908800ULL` (OFF).
- **Behavior**:
  - If the spork is active: blocks are built as **Version 12** under Standard Mode rules.
  - If the spork is inactive: blocks are built as **Version 11** under Fallback Mode rules.

Network Configurations & Activation Heights

The transition of the cooperative consensus framework consists of three activation thresholds:

1. **`Consensus::UPGRADE_ADAM`**: Activates ADAM Fallback Mode (Block Version 11).
2. **`Consensus::UPGRADE_POMBL`**: Activates ADAM Standard Mode / Multi-Proof-Algorithm (Block Version 12).
3. **`Consensus::UPGRADE_MODELD`**: Enforces Model D reward splits and PoBLS consensus.

Network	**`UPGRADE_ADAM`** (ADAM & Dual-Track Start)	**`UPGRADE_ADAM_V2`** (Shift V2)	**`UPGRADE_POMBL`** (Standard Mode / V12)	**`UPGRADE_MODELD`** (Model D / PoBLS)	Target Spacing
Mainnet	Block 200 (`nAdamDualTrackHeight = 200`)	Block 705	Block 2000	Block 2200	30 seconds
Testnet	Block 200 (`nAdamDualTrackHeight = 200`)	Block 300	Block 400	Block 500	30 seconds
Regtest	Block 200 (`nAdamDualTrackHeight = 1`)	Block 705	Block 300	Block 200	10 seconds

D. Puzzle Difficulty Bit-Shift Parameters

To prevent a “false-positive flood” of puzzle solutions on the network while maintaining dynamic and configurable difficulty scaling across different block heights, ADAM utilizes `GetAdamDifficultyShift(nHeight, consensus)` (`src/adam.cpp`) governed by `Consensus::Params`:

- **V1 Difficulty Shift (`nAdamDifficultyShiftV1`)**: Bit-shift multiplier relaxing target difficulty for block heights before `Consensus::UPGRADE_ADAM_V2`. Value: `10`.
- **V2 Difficulty Shift (`nAdamDifficultyShiftV2`)**: Bit-shift multiplier relaxing target difficulty for block heights at or above `Consensus::UPGRADE_ADAM_V2`. Value: `6`.
- **Upgrade Activation (`Consensus::UPGRADE_ADAM_V2`)**: The network upgrade height at which the difficulty shift transitions from `V1` (`10`) to `V2` (`6`) (`705` on Mainnet/Regtest, `300` on Testnet).

During puzzle verification, the `scaledTarget` is derived by shifting the consensus target (`Target`) by the current active difficulty shift value:

$$\text{scaledTarget} = \text{Target} \ll \text{activeShift}$$

- activeShift=nAdamDifficultyShiftV1 (`10`) if `NetworkUpgradeActive(nHeight, consensus, Consensus::UPGRADE_ADAM_V2)` is `false`.
- activeShift=nAdamDifficultyShiftV2 (`6`) if `NetworkUpgradeActive(nHeight, consensus, Consensus::UPGRADE_ADAM_V2)` is `true`.

E. Starting Difficulty Limit (powLimit)

To prevent blocks 1–199 from being mined too quickly (which led to split forks and quorum deadlocks), the starting difficulty target `powLimit` is set to:

$$\text{powLimit} = \text{UINT256_ZERO} \gg 20$$

On Mainnet and Testnet, this is exactly `1/2^20` (equivalent to the genesis block’s `nBits` of `0x1e0ffff0`). It ensures blocks are naturally spaced out at approximately 30 seconds from genesis, allowing nodes to establish stable P2P connections and maintain a unified chain tip.

---

3. Verifiable Random Function (VRF) & Rolling Seeds

To prevent **grinding attacks** (where miners alter transactions or nonces to manipulate the hash of block H, thereby skewing the leader election for block $H+1$), ADAM implements a **Verifiable Random Function (VRF) Rolling Seed** model.

Mathematical Formulation

For any block height H where the ADAM network upgrade (`Consensus::UPGRADE_ADAM`) is active:

$$\text{Seed}_H = \text{Hash}(\text{Seed}_{H-1} \| \text{VRFProof}_{H-1})$$

Where:

- The election seed Seed at height H is used to determine the leaders for block $H+1$.

- The VRF proof VRFProof at height $H-1$ is the signature generated by the elected **Coordinator** of block $H-1$ on the previous seed at height $H-2$ using a hybrid BLS12-381 + ECDSA fallback signature mechanism (`SignBLSWithECDSAFallback`).
- Under this scheme, the Coordinator generates a BLS12-381 signature using a BLS secret key derived deterministically from their ECDSA private key. To prove authorization and prevent key manipulation, the Coordinator signs the BLS public key using their ECDSA private key. Verification (`VerifyBLSWithECDSAFallback`) ensures both the ECDSA authorization of the BLS key and the BLS signature of the target seed are valid.
- Because the resulting signature is deterministic, the Coordinator has exactly one valid signature for a given seed, making the rolling seed completely immutable and unpredictable until the Coordinator signs and broadcasts it.

Node Selection (SSLE)

The election of miners and coordinator is performed by `SelectAdamNodes()` inside `src/adam.cpp`:

1. Compile the active node pool (the registered Masternode list and active registered miners via Coin-Lock or PoW-Lock).

2. The selection pool is network-dependent:

   - **Mainnet & Testnet**: The pool is constructed dynamically from active Masternodes and active registered miners. However, to ensure stable block production before active masternodes or registrations are established, as long as the block height is below the bootstrap limit (`nAdamBootstrapLimit` = 44,850 on Mainnet or 100,000 on Testnet), the network automatically scans the block producers (coinbase outputs) from blocks 1 to 199 and adds their public keys to the selection pool. This prevents chain stalls before active masternodes or registrations are established.
   - **Regtest**: The pool automatically bypasses external registrations and includes 15 deterministic bootstrap public keys to facilitate automated testing:

     $\text{Pool}_{\text{bootstrap}} = \{\text{DeterministicPubKey}_0, \ldots, \text{DeterministicPubKey}_{14}\}$

3. **Compute a unique hash rank for each node in the selection pool based on the rolling seed**:

   $\text{Rank}_i = \text{Hash}(\text{Seed}_H \| \text{PubKey}_i)$

4. **Sort the pool in ascending order of their** Rank_i.

5. **Dual-Track (**$8+3$**) Committee Election (`nAdamDualTrackHeight` = 200 on Mainnet/Testnet, 1 on Regtest; must always be set to a future block height** $\geq$UPGRADE_ADAM **when activated on an existing chain)**: When `IsAdamDualTrackActive(H)` is true (`H >= consensus.nAdamDualTrackHeight`), the 11 validator seats are partitioned into two tracks via `ElectDualTrackValidators()` with bidirectional dynamic spillover:

   - **Track 1 — Collateralized Track (`ADAM_COLLATERALIZED_TRACK_QUOTA = 8` seats)**:
     - **Sub-Track 1A (`ADAM_MASTERNODE_TRACK_QUOTA = 7` seats)**: Reserved for **Active Masternodes (`4,200 KRISTA` collateral)**. Enforced at compile time via `static_assert(ADAM_MASTERNODE_TRACK_QUOTA >= 7)` so Active Masternodes alone can always form a post-bootstrap quorum (`GetAdamThreshold = 7`).
     - **Sub-Track 1B (`ADAM_COIN_LOCK_TRACK_QUOTA = 1` seat)**: Reserved for `250 KRISTA` **Coin-Lock** validators.

- Immediate intra-Track-1 spillover between Sub-Track 1A and Sub-Track 1B ensures that when no Coin-Locks exist, Active Masternodes fill all 8 seats of Track 1 (`7 + 1 = 8`), and when Active Masternodes `< 7`, Coin-Locks fill the remainder of Track 1 before spilling over to Track 2.

- **Track 2 — Open PoW Track (`ADAM_OPEN_POW_TRACK_QUOTA = 3` seats)**: Reserved for Open PoW miners (`PoW-Lock` with `10 KRISTA` CLTV lock, `Zero-Coin PoW`, and `Block 1..199` bootstrap keys). If fewer than 3 Open PoW candidates exist, unfilled seats in Track 2 spill over first to remaining Active Masternodes and then to remaining Coin-Locks.
- This guarantees that all 11 validator seats are always filled while protecting `4,200 KRISTA` Active Masternodes (`7` validator seats + `1` Coordinator), giving `250 KRISTA` Coin-Locks a dedicated `1`-seat collateralized sub-track isolated from `10 KRISTA` `PoW-Lock` dilution, and capping Open PoW at `3` seats (`27.27%`) when the collateralized tracks are populated.
- **Standby Validator Seat Handover & Time-Relaxed Quorum Threshold (`ResolveAdamCommitteeForTime`, `ValidateAdamCommitteeForTime`, `GetAdamEffectiveThreshold`)**:
  - If a block is delayed by ≥ 60s (`ADAM_COMMITTEE_FALLBACK_TIMEOUT_SECS = 60`, advancing in `ADAM_COMMITTEE_FALLBACK_SLOT_SECS = 30` slots), any silent seat in the 11-validator committee that has not submitted a solution may be filled ONLY by a **same-track** standby validator (`MN -> MN`, `Coin-Lock -> Coin-Lock`, `Open PoW -> Open PoW`) from `vStandbyMiners` in strict sequential rank order within that track (`k_t > prevStandbyIdxInTrack[t]` and `k_t < nReplacedInTrack[t] + nSlot`), without displacing any honest primary validator that already solved its puzzle. Both `ResolveAdamCommitteeForTime` and `ValidateAdamCommitteeForTime` resolve validator tracks against the block's `CCoinsViewCache* pView` so reorg and initial-sync validation match `SelectAdamNodes` identically.
  - Under Model D (`IsModelDActive`), **Equal Work, Equal Reward** applies across all 11 committee seats: every seat $i \in \{0, \ldots, 10\}$ that provides a valid PoW solution in `vAdamSolutions[i]` — whether filled by the original primary validator `vExpectedMiners[i]` OR by a legitimate same-track standby validator `vAdamMiners[i]` when the primary seat timed out ($t \geq 60$s) — receives its full track-specific share of the 40% Participant Validator reward pool (20% across the 7 Masternode Validator quota seats at `~2.857%` each + 20% across the 4 Independent Miner quota seats [`8.00%` for 1 Coin-Lock + `4.00%` each for 3 Open PoW]). Unsolved/empty seats (`vAdamSolutions[i].empty()`) receive `0%`, while the Coordinator retains its full `15%` reward (`nProducerSplit`) when rescuing a stalled block with standby validators. Any unclaimed empty-seat shares (`nUnclaimedPartSplit = (nPartSplitTotal - nPaidPartTotal)`) are split **50% to the Automatic Protocol Burn (`nExpectedBurn = nUnclaimedPartSplit / 2`)** and **50% to the Developer Treasury (`nExpectedTreasury = nTreasurySplit + (nUnclaimedPartSplit - nExpectedBurn)`)** while the Passive Masternode Winner receives its exact `35%` (`nMNSplit`). Furthermore, during LLMQ block proposal validation (`qblockprop`, evaluated only when the node's current tip matches `msg.block.hashPrevBlock`), if $t \geq 60$s and a primary seat i is **substituted** by a standby key (`vAdamMiners[i] != vExpectedMiners[i]`) while the LLMQ member already holds a valid cached solution from `vExpectedMiners[i]`, the LLMQ member rejects the proposal and refuses to sign `QUORUMSIGSHARE` (empty seats `vAdamMiners[i] == vExpectedMiners[i]` are not vetoed so normal fast blocks are never delayed by in-flight P2P race conditions).
  - If a block remains stalled for ≥ 120s (`ADAM_THRESHOLD_RELAXATION_TIMEOUT_SECS = 120`), `GetAdamEffectiveThreshold(nHeight, nTimeElapsed, nActiveMasternodes)` relaxes the required quorum threshold by `1` every `60s` (`ADAM_THRESHOLD_RELAXATION_SLOT_SECS = 60`) down

to `ADAM_MIN_RELAXED_THRESHOLD = 5` (`> 1 Coin-Lock + 3 Open PoW = 4`, enforced via `static_assert` and runtime `assert` so at least 1 Active Masternode is always required) whenever the block is post-bootstrap (`nHeight >= nAdamThresholdBootstrapLimit`) OR active Masternodes in the pool are ≥ 7. Only during the bootstrap period (`nHeight < nAdamThresholdBootstrapLimit`) when fewer than 7 Active Masternodes exist does the floor relax to `ADAM_MIN_RELAXED_THRESHOLD_BOOTSTRAP = 3`. Header-only validation (`ContextualCheckBlockHeader`) passes `0` for `nActiveMasternodes` (since the block's UTXO view is not yet available at header check time, avoiding false bans during sync/reorg), while full block validation (`CheckFormsOfWork`) strictly enforces `GetActiveMasternodePoolCount(nHeight - 1, pView)` against the block's actual UTXO view.

- Node-wide version-independent single-block-per-parent guard (`ActiveProposalGuard` in `src/miner.cpp`) prevents any node from concurrently or sequentially producing both a PoW block and a PoS block for the same `(parentHash, coordinatorKey)` across all block versions (`v11` and `v12`).

6. **Coordinator Election**:

- **Rule 1 (`Active Masternodes > Threshold`)**: The highest-ranked Active Masternode is elected as the **Coordinator**, and the 11 **Miners** are elected from the remaining pool via `ElectDualTrackValidators`.
- **Rule 2 (`Active Masternodes <= Threshold`)**: The 11 **Miners** are elected via `ElectDualTrackValidators`, and the highest-ranked non-elected key in the pool is elected as the **Coordinator**.

---

4. Block Header Extensions & Serialization

When the ADAM network upgrade (`Consensus::UPGRADE_ADAM`) is active, blocks are serialized using **Version 11** (Fallback Mode) or **Version 12** (Standard Mode) block structures. The `CBlockHeader` class in `src/primitives/block.h` is extended with four new fields:

```
class CBlockHeader {
public:
    // Legacy fields...
    int32_t nVersion;
    uint256 hashPrevBlock;
    uint256 hashMerkleRoot;
    uint32_t nTime;
    uint32_t nBits;
    uint32_t nNonce;

    // ADAM Extended fields (Version >= 11)
    std::vector<CPubKey> vAdamMiners;                // Public keys of elected miners (and
        coordinator in v11)
    std::vector<std::vector<unsigned char>> vAdamSolutions; // Serialized partial solutions
        (nonce + miner signature)
    std::vector<unsigned char> vAdamVRFProof;        // Coordinator's VRF signature on
        previous seed
    std::vector<unsigned char> vAdamCoordinatorSig;  // Coordinator's signature on final
        block header hash
};
```

4.1 Hashing and Serialization Order

Unlike `vAdamCoordinatorSig` (which signs the finished block header hash and is excluded from `GetHash()`), `vAdamVRFProof` is generated *before* the block hash is computed. It is included in the block header serialization during `GetHash()`, ensuring the VRF proof is cryptographically locked into the header.

4.2 Block Hashing Chain (Stateless)

To calculate the block hash (`CBlockHeader::GetHash()`), ADAM hashes the serialized header (excluding `vAdamCoordinatorSig` and the LLMQ `vQuorumSig` if version 12) through a chain of M sequential hashing rounds corresponding to the elected miners:

- In fallback mode (`nVersion == 11`): M = vAdamMiners.size() − 1.
- In standard mode (`nVersion == 12`): M = vAdamMiners.size().

For each round $i \in \{0, \ldots, M-1\}$:

1. Derive a unique round hash from the previous block hash and round index:

   $\text{roundHash}_i = \text{Hash}(\text{hashPrevBlock} \| i)$

2. Extract the first byte $v_i = \text{roundHash}_i[0]$.

3. Calculate an odd coprime multiplier m_i:

   $m_i = v_i \mid 1$

   If $m_i < 3$, set $m_i = 3$. This ensures the multipliers are coprime to 2^{256}, preserving 100% entropy.

4. Perform the round hashing:

   - **Fallback Mode (Version 11)**:
     - **Round 0**: $H_0 = \text{CalculateAdamPuzzleHash}(\text{algo}_0, \text{SerializedHeader}) \times m_0 \pmod{2^{256}}$
     - **Round $i > 0$**: $H_i = \text{CalculateAdamPuzzleHash}(\text{algo}_i, H_{i-1}) \times m_i \pmod{2^{256}}$
   - **Standard Mode (Version 12)**:
     - **Round 0**:
       - Derive 3-permutation algorithms algo1, algo2, algo3 for miner 0.
       - Compute:

         $H_0^{(3)} = \text{CalculateAdamPuzzleHash}(\text{algo3}, \text{SerializedHeader})$

         $H_0^{(2)} = \text{CalculateAdamPuzzleHash}(\text{algo2}, (H_0^{(3)} \times 1) \pmod{2^{256}})$

         $H_0 = \text{CalculateAdamPuzzleHash}(\text{algo1}, (H_0^{(2)} \times 1) \pmod{2^{256}})$

       - Apply multiplier:

         $H_{\text{prev}} = (H_0 \times m_0) \pmod{2^{256}}$

     - **Round $i > 0$**:

- Derive 3-permutation algorithms algo1, algo2, algo3 for miner i.
- Compute:

  $H_i^{(3)}$ = CalculateAdamPuzzleHash(algo3, H_{prev})

  $H_i^{(2)}$ = CalculateAdamPuzzleHash(algo2, ($H_i^{(3)} \times (i + 1)$) (mod 2^{256}))

  H_i = CalculateAdamPuzzleHash(algo1, ($H_i^{(2)} \times (i + 1)$) (mod 2^{256}))
- Apply multiplier:

  $H_{\text{prev}} = (H_i \times m_i) \pmod{2^{256}}$

The final hash H_{prev} (or $H_0 \times m_0$ in Version 11) is returned as the block hash.

---

5. Consensus Validation Rules

When a block is received, `CheckBlock()` in `src/main.cpp` enforces the following validations if the ADAM network upgrade (`Consensus::UPGRADE_ADAM`) is active:

1. **Version Enforcement & Downgrade Prevention**:
   - The block version must be at least `11`.
   - If `block.nVersion == 11` and `block.GetBlockTime() >= sporkManager.GetSporkValue(SPORK_21_ADAM_STANDARD_MODE)`, the block is rejected with a `bad-version` DoS error to prevent malicious miner downgrades.
   - If the block height is ≥nPoMBLHeight (300 on Regtest, 400 on Testnet, 2000 on Mainnet), the block version must be `12` if Standard Mode is active.
2. **Miners & Solutions Sizing**:
   - **Version 11 (Fallback Mode)**:
     - `vAdamMiners` size must be between 11 and 14.
     - `vAdamSolutions` size must be exactly `vAdamMiners.size() - 1`.
     - The Coordinator to verify is the last element of `vAdamMiners` (`vAdamMiners.back()`).
   - **Version 12 (Standard Mode)**:
     - `vAdamMiners` size must match exactly `nAdamMinersCount`.
     - `vAdamSolutions` size must match exactly `vAdamMiners.size()`.
     - The Coordinator is derived from `SelectAdamNodes`.
3. **VRF Proof Validation**: The `vAdamVRFProof` must be verified against the previous rolling seed Seed_{H-1} and must be a valid signature matching the public key of the expected Coordinator using `VerifyBLSWithECDSAFallback`.

4. **Election Path Validation**: In Version 12, the list of public keys in `vAdamMiners` must match the exact output of the deterministic `SelectAdamNodes` algorithm. Version 11 bypasses this check since it operates in self-contained bootstrap mode.

5. **Partial Solutions Validation**:

   - The number of valid partial solutions must meet the required threshold governed by `GetAdamThreshold(nHeight)`:
     - **Version 11 & Version 12**: 5 valid solutions during the bootstrap phase (block height < 50,000 on Mainnet, < 100,000 on Testnet) to ensure uninterrupted liveness, transitioning to the full consensus parameter `nAdamThreshold` (7 solutions on Mainnet/Regtest, 3 solutions on Testnet) at block 50,000.
   - Each solution is parsed into a `nonce` and a `signature`.
   - The puzzle hash is calculated using the algorithm(s) assigned to the miner:
     - In **Version 11 (Fallback Mode)**:

       PuzzleHash = CalculateAdamPuzzleHash(algoIndex, $\text{Seed}_H \| \text{MinerPubKey}_i \| \text{Nonce}_i$)

       where algoIndex = Hash($\text{Seed}_H \| \text{MinerPubKey}_i$) (mod 18), using one of the **18 supported algorithms** (including `Hamsi`, `Fugue`, `Shabal`, `Whirlpool`, and `Haval-256`).

     - In **Version 12 (Standard Mode)**: Uses a 3-permutation selector scheme GetAdam3PermutationAlgos(hashPrevBlock, MinerPubKey_i) that deterministically selects 3 distinct hashing algorithms (algo1, algo2, and algo3) out of 18 available algorithms based on the previous block's hash and the miner's public key. The solver compounds the three algorithms:

       PuzzleHash = $\text{algo1}((\text{minerIdx} + 1) \times \text{algo2}((\text{minerIdx} + 1) \times \text{algo3}(\text{Challenge}))) \pmod{2^{256}}$

       where Challenge = $\text{Seed}_H \| \text{MinerPubKey}_i \| \text{Nonce}_i$.

   - The `PuzzleHash` must satisfy the target difficulty defined by `nBits` (relaxed to `scaledTarget = Target \ll \text{activeShift}`, as described in Section 2.D).
   - The signature must be verified against `MinerPubKey_i` signing the `PuzzleHash`.

6. **Coordinator Signature Validation**: The `vAdamCoordinatorSig` must be verified against the expected Coordinator's public key signing the final block header hash (excluding the signature itself) using `VerifyBLSWithECDSAFallback`.

---

6. Wallet & RPC API Integration

To support developers and mining pool operators, the RPC API exposes the cooperative consensus fields:

`getblocktemplate` Request & Response

External miners (`cpuminer-opt -a adam`, `ccminer -a adam`) may pass a JSON template request object containing `"mineraddress"` (`{"mineraddress": "<KRISTA_address>"}`) to target a specific registered validator or Masternode operator seat held by the wallet. When `nVersion >= 11`, the JSON response includes:

- `version`: `11` (Fallback Mode) or `12` (Standard PoMBL Mode)
- `status`: `"waiting_for_election"` if `"mineraddress"` was specified and that validator key is not elected in the current round (preventing wasted CPU/GPU cycles), otherwise active puzzle work fields (`minerindex`, `minerpubkey`, `puzzlealgo` / `puzzlealgos`, `puzzletarget`, `seed`).
- `adamminers`: Array of hex-encoded public keys of the elected miners.
- `adamvrfproof`: Hex-encoded VRF signature on the previous block's seed.
- `adamsolutions`: Array of hex-encoded partial solutions currently collected.
- `adamcoordinatorsig`: Hex-encoded signature of the Coordinator.

`generate` RPC & Qt Dashboard Miner Hub

The `generate` / `setgenerate` RPCs and the **KristaTech-Qt Dashboard Miner Hub** handle cooperative block assembly and puzzle solving for ADAM blocks. Instead of performing traditional high-difficulty hash grinding, the node:

1. Determines which local wallet/Masternode keys are elected as Committee Validators or Coordinator based on the rolling seed.
2. Solves lightweight partial PoW puzzles across all locally held elected validator seats and broadcasts `ADAMSOL` messages.
3. If elected as Coordinator, collects committee solutions, deterministically signs the VRF proof and final block hash (requesting a 3-of-5 LLMQ `vQuorumSig` once Model D is active), and submits the finished block to the network. The Qt Dashboard Miner Hub exposes 1-click CPU mining, non-blocking 1-click `PoW-Lock` / `Coin-Lock` registration, live committee role badges, and 1-click masked `cpuminer` command copy.

---

7. Proof-of-Stake (PoS) Integration & Cooperative PoS

ADAM does not replace Proof-of-Stake (PoS) but integrates with it to form a **Hybrid Cooperative PoS** consensus mechanism where PoS and PoW (ADAM cooperative mining) function together.

In traditional PoS, block production is determined solely by the staking weight (the amount of coins held in a wallet). In ADAM, this is combined with the cooperative miner-coordinator validation loop to prevent block grinding, selfish staking, and targeted leader DoS.

The Staking and Cooperative Lifecycle (Block height $\geq$ 200)

Once the network upgrade `Consensus::UPGRADE_POS` activates (at block height 200 on Mainnet), block generation transitions from pure Cooperative PoW to Hybrid Cooperative PoS, where PoW cooperative mining and PoS staking run in parallel:

> **IMPORTANT**
>
> **No Single-Signature PoS Blocks:** Proof-of-Work (ADAM cooperative mining) never ends or gets disabled in favor of PoS. Every PoS block in KristaTech is hybrid, requiring the validation of PoW puzzles solved by elected ADAM miners, and must be double-signed by the staker's key (`vchBlockSig`) and the elected ADAM coordinator's key (`vAdamCoordinatorSig`). Single-signature blocks are strictly rejected by consensus validation rules.

1. **Staking Entitlement (Kernel Check)**: The wallet's staking thread (`ThreadStakeMinter`) periodically evaluates if any UTXOs are eligible to stake a block by verifying the kernel hash check (proportional to coin weight).

2. **Cooperative Puzzle Collection**: Once a staking thread wins the right to propose a block, it builds a block template (Version 12, as both PoS and `UPGRADE_POMBL` are active at block height ≥ 2000). The staker's wallet retrieves the elected miners for the current block height via `SelectAdamNodes` and collects the lightweight PoW puzzles solved by these elected miners from the P2P network memory cache (`mapAdamSolutionsCache`). If any elected miner's solution is missing from the local cache, the block template is deferred until all required solutions are received.

3. **Coordinator Validation and Signature**: The elected Coordinator of the current round validates the block template, signs the previous block's seed to produce the VRF proof (`vAdamVRFProof`), and signs the block header to generate the Coordinator signature (`vAdamCoordinatorSig`).

4. **Staker Block Signature (Dual Locking)**: The finalized block is secured using two distinct signature types:

   - **ADAM Signature (`vAdamCoordinatorSig`)**: Generated by the elected Coordinator to validate that the cooperative PoW mining round was completed successfully.
   - **PoS Block Signature (`vchBlockSig`)**: Generated by the staker's wallet using `SignBlock` (signing the final block hash using the private key of the staking UTXO).

Dual Validation on the Network

When a peer receives a Cooperative PoS block, the validation rules in `CheckBlock` require both consensus checks to pass:

1. **Proof-of-Stake Verification**: The node verifies the `coinstake` transaction, checks the kernel hash target difficulty, and verifies the staker's block signature (`vchBlockSig`).
2. **ADAM Cooperative PoW Verification**: The node verifies the elected miners' partial PoW puzzle solutions (`vAdamSolutions` meeting the quorum threshold T), the Coordinator's VRF proof (`vAdamVRFProof`), and the Coordinator's signature (`vAdamCoordinatorSig`), as well as the LLMQ quorum signature (`vQuorumSig`) in Version 12 blocks once Model D is active.

---

8. LLMQ (Long-Living Masternode Quorums) & DKG (Distributed Key Generation)

To validate and sign Version 12 block templates under Standard Mode, the network relies on **Long-Living Masternode Quorums (LLMQs)** which execute **Distributed Key Generation (DKG)** sessions.

> **NOTE**
> **Quorum Threshold Distinction:** Do not confuse the **ADAM Cooperative Mining Threshold** (7 out of 11 rule for puzzle solutions `vAdamSolutions`) with the **LLMQ Signature Verification Threshold** (3 out of 5 rule for block signature `vQuorumSig`):
>
> - **ADAM Threshold**: Enforces that at least 7 out of 11 elected miners (`nAdamThreshold = 7` on Mainnet/Regtest, `3` on Testnet; `5` during the initial bootstrap period below `nAdamThresholdBootstrapLimit`) must solve and submit their PoW puzzle solutions (`vAdamSolutions`) for the block to be accepted. This is part of the ADAM cooperative puzzle validation.
> - **LLMQ Threshold**: Enforces that at least 3 out of 5 LLMQ members (`quorum.members.size() / 2 + 1`) must sign the proposed block hash using their secp256k1 private keys (`vQuorumSig`). This is part of the decentralized block validation.

A. Quorum Topology and Parameters

- **Quorum Size**: Every active LLMQ consists of exactly **5 members** (`llmq.cpp:218`).
- **DKG Intervals & Per-Block Rotation**:
  - **Initial Epoch Mode**: A new DKG session is executed every **100 blocks** on Mainnet and every **10 blocks** on Testnet/Regtest (`GetActiveQuorum`).
  - **Per-Block Quorum Rotation**: Once quorum rotation activates (at block height **1,400** on Mainnet, **1,000** on Testnet, and **800** on Regtest), `GetActiveQuorum` runs `RunDKG(nHeight)` at each block height using the epoch seed from `((nHeight / dkgInterval) * dkgInterval) - 1`.
- **Quorum Signature Verification Threshold (`CQuorumSignature::Verify`)**:
  - **All Networks (Mainnet, Testnet, Regtest)**: Once Model D (`UPGRADE_MODELD`) is active, `CQuorumSignature::Verify` enforces a BFT majority threshold of `quorum.members.size() / 2 + 1` (minimum 2, capped at `quorum.members.size()`), which requires at least **3 out of 5** quorum signatures.
  - **Prior to Model D Activation**: On Mainnet (blocks 2,000–2,199) and Testnet (blocks 400–499), quorum signature verification is bypassed in `CheckBlock()` (`fFallbackMode`) to allow initial masternode accumulation. On Regtest, Model D activates at height 200 (before `UPGRADE_POMBL` at height 300), so the 3-of-5 majority threshold is active from block 300 onward.

B. Active Masternode Filtering (100% On-Chain Deterministic Pool)

To ensure the network is robust, decentralized, and immune to P2P memory-state divergence:

- Quorums and ADAM validator elections are resolved 100% deterministically from verified on-chain Masternode state (`mapMasternodeCollateralOutpoints`, `mapOnChainCollateralDelegations`, and 4,200 KRISTA collateral verification in `pcoinsTip`), with zero fallback to in-memory P2P `mnodeman`.

- If fewer than 5 on-chain active masternodes are registered, the network deterministically falls back to electing quorum members from the registered miner pool (`GetAdamMinerPool(nHeight - 1)` — mined blocks 1-199 bootstrap or coin-lock/pow-lock registrations).
- No hardcoded key ID filtering or local wallet restrictions are applied on Mainnet, Testnet, or Regtest, ensuring a fully decentralized and trustless consensus environment.

C. Sliding Window Quorum Rotation

To protect the quorum members against targeted Denial-of-Service (DoS) and DDoS attacks, a **Sliding Window Quorum Rotation** mechanism is implemented. Under this scheme:

- Instead of using a fixed static quorum or electing a completely new set of members each epoch, the quorum members are selected from the active pool of masternodes using a sliding offset based on the block height:

  $\text{member_index}_i = (\text{nHeight} + i) \pmod{\text{total_masternodes}}$

- At each new block height `nHeight`, the offset slides by 1, meaning that exactly **4 out of 5 members** are preserved from the previous block (retaining consensus continuity and DKG handshake state) and exactly **1 member is rotated**.

- This makes targeted attacks economically unviable and highly complex, as the active members shift constantly before an attacker can map and flood them.

The rotation upgrade activates at target upgrade block heights per network:

- **Regtest**: Block height ≥ 800
- **Testnet**: Block height ≥ 1000
- **Mainnet**: Block height ≥ 1400

---

9. Offline Coordinator Fallback Rotation

To prevent network freezes during the bootstrap phase (Fallback Mode, Block Version 11, active for block heights < 2000) when the primary elected coordinator goes offline:

A. Fallback Activation Delay

If the elapsed time since the previous block time exceeds **45 seconds** (`ADAM_FALLBACK_COORDINATOR_TIMEOUT_SECS = 45`), the stakers and miners determine that the primary elected coordinator is offline or unreachable.

B. Rotation Protocol

Once fallback activation is triggered, the coordinator role is rotated among the **11 elected miners** (`vExpectedMiners` , rank 0 to 10) every **30 seconds**:

$$\text{rotationIndex} = \left(\frac{\text{timeElapsed} - 45}{30}\right) \pmod{11}$$

The staker or miner that owns the private key corresponding to `vExpectedMiners[rotationIndex]` dynamically takes over block template construction, VRF signing, and block header broadcasting.

C. Consensus Compatibility

Because Version 11 block validation rules (`CheckBlock` in `src/main.cpp`) do not enforce that the coordinator must match the primary elected coordinator (it only verifies the signature and VRF validity of the coordinator public key listed in the block), this fallback coordinator rotation successfully allows the network to bypass offline leaders and progress without consensus breaks or hard forks.

D. Standard Mode Strict Verification (Block Version 12)

In Standard Mode (Block Version 12, active for block heights ≥ 2000), strict deterministic coordinator selection is ordinarily enforced on-chain. The coordinator must match the expected coordinator elected by `SelectAdamNodes`.

However, to prevent network freezes when the number of active, enabled masternodes drops below a critical threshold (specifically < 11), the network dynamically activates **Fallback Mode** for block heights where the Model D upgrade is active (≥ 2200 on Mainnet, ≥ 500 on Testnet, or ≥ 200 on Regtest, verified via `IsModelDActive`) (referred to as **Masternode Fallback Mode / Yöntem B #2**).

Under Masternode Fallback Mode:

1. Block validation rules switch to **Fallback validation rules** (allowing fallback coordinator rotation and checking for M = vAdamMiners.size() − 1 solutions).
2. The block template is built by appending the coordinator key to the end of `vAdamMiners`.
3. If the elected coordinator is offline, the fallback coordinator rotation protocol (Section 9.B) is activated for Version 12 blocks as well, dynamically rotating the coordinator among the elected miners.

This dual-mode consensus rule ensures the network remains self-healing and high-availability even during severe masternode outages.

---

10. Consensus Finality, Fork Resolution & Performance Hardening (v1.4.5.9)

A. Deterministic Equal-Work Fork Tie-Breaker (`CBlockIndexWorkComparator`)

In a 30-second cooperative PoW/PoS hybrid chain, network partitions or simultaneous fallback rotations can occasionally produce competing blocks at the same height with identical cumulative chain work (`nChainWork`). Previously, `CBlockIndexWorkComparator` in `src/chain.h` fell back to sequence ID (`nSequenceId` — order of local receipt) before comparing block hashes, causing partitioned subsets of nodes to remain stuck on different equal-work tips until the next block was mined.

- **Rule**: When `pa->nChainWork == pb->nChainWork`, the comparator deterministically prefers the block index with the numerically smaller block hash (`pa->GetBlockHash() < pb->GetBlockHash()`) before checking `nSequenceId`.
- **Effect**: All peers immediately converge on the exact same canonical chain tip (`< 1s` convergence time) as soon as competing blocks propagate across a healed partition.

B. LLMQ Anti-Equivocation Slot Guard & Multi-Hop Gossip Relay

- **Per-Slot Anti-Equivocation (`mapQuorumSignedSlot`)**: In `src/llmq.cpp`, each LLMQ quorum member tracks `mapQuorumSignedSlot[std::make_pair(nHeight, rotationIndex)]`. Once a quorum member signs a valid `PROPOSEBLOCK` for a given `(nHeight, rotationIndex)` slot, it refuses to sign any competing block proposal for the same slot unless the coordinator rotation index advances. This guarantees that two competing blocks in the same slot cannot both collect a 3-of-5 quorum certificate (`vQuorumSig`).
- **Multi-Hop Gossip Relay (`mapSeenBlockProposals`, `mapSeenSigShares`)**: `PROPOSEBLOCK` and `QUORUMSIGSHARE` P2P messages are deduplicated and relayed across all connected peers, ensuring quorum members and block proposers receive proposals and signature shares even when not directly connected in a single hop.
- **100% On-Chain Fallback in `ElectQuorumMembers`**: When the number of Masternodes with recent on-chain heartbeat pings (within the 2,880-block activity window) and unspent 4,200 KRISTA collateral is less than `nQuorumSize` (5), `ElectQuorumMembers` deterministically falls back to the on-chain `GetAdamMinerPool(nHeight - 1, pView)` candidate pool without touching in-memory P2P `mnodeman` state.

C. Authenticated On-Chain Masternode Heartbeat Pings (`ExtractOnChainMasternodePing` & `SendOnChainPing`)

- **Compressed (33-byte) and Uncompressed (65-byte) Public Key Support**: `ExtractOnChainMasternodePing` in `src/adam.cpp` validates both 36-byte (`0x21 <33-byte compressed pubkey> OP_CHECKSIGVERIFY OP_1`) and 68-byte (`0x41 <65-byte uncompressed pubkey> OP_CHECKSIGVERIFY OP_1`) MESCAL `check-signature-verification` heartbeat scripts alongside the serialized `CDataStream` `OP_RETURN` proof (`collateralOutpoint` + `vchPingSig` over `Hash(collateralOutpoint || tx.vin[0].prevout)` + optional `bool hasDelegation` + `COnChainMasternodeDelegation`).
- **Relay Fee Exemption (`0.0001 KRISTA`)**: Authenticated on-chain Masternode ping transactions are exempt from the `1000x` `OP_RETURN` relay fee multiplier in `GetMinRelayFee` (`src/main.cpp`), allowing Hot Nodes to broadcast periodic on-chain heartbeat pings for a flat `0.0001 KRISTA` fee.

D. Lock-Free Chain Tip Access (`CChain::AtomicTip()`)

- `CChain` (`src/chain.h` / `src/chain.cpp`) maintains an atomic pointer `std::atomic<CBlockIndex*> pAtomicTip` updated inside `CChain::SetTip()`.
- `GetChainTip()` (`src/main.cpp`) reads `chainActive.AtomicTip()` without acquiring `cs_main`, eliminating lock contention between P2P message processing, RPC queries, and staking/mining threads.